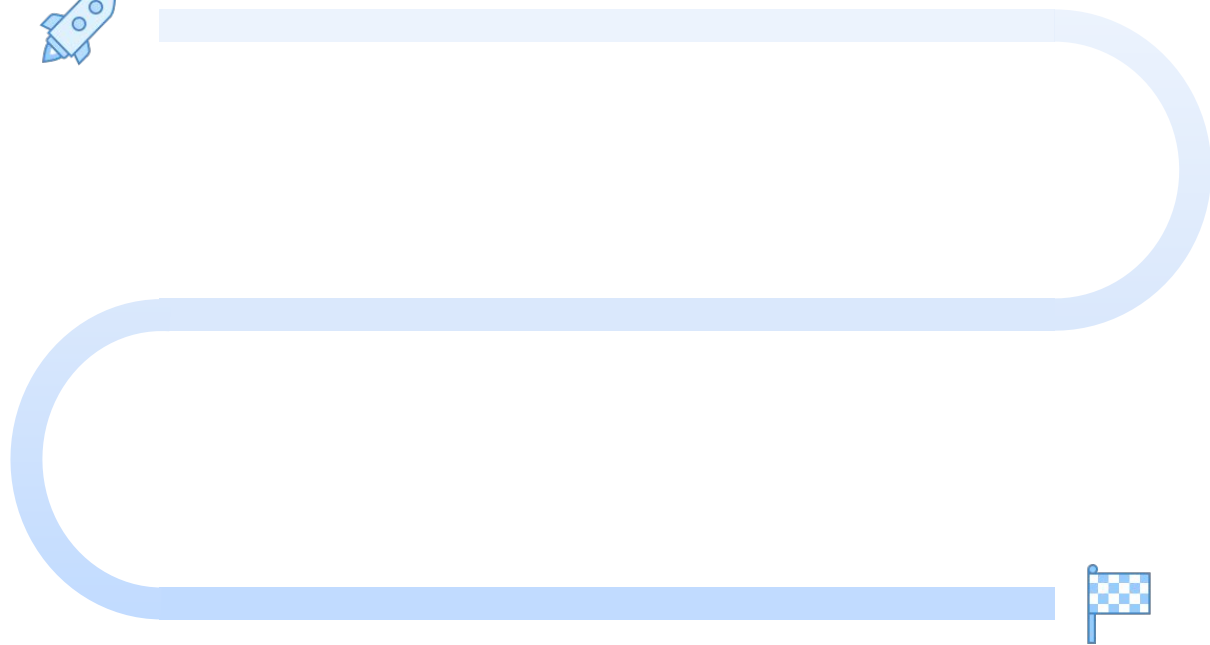
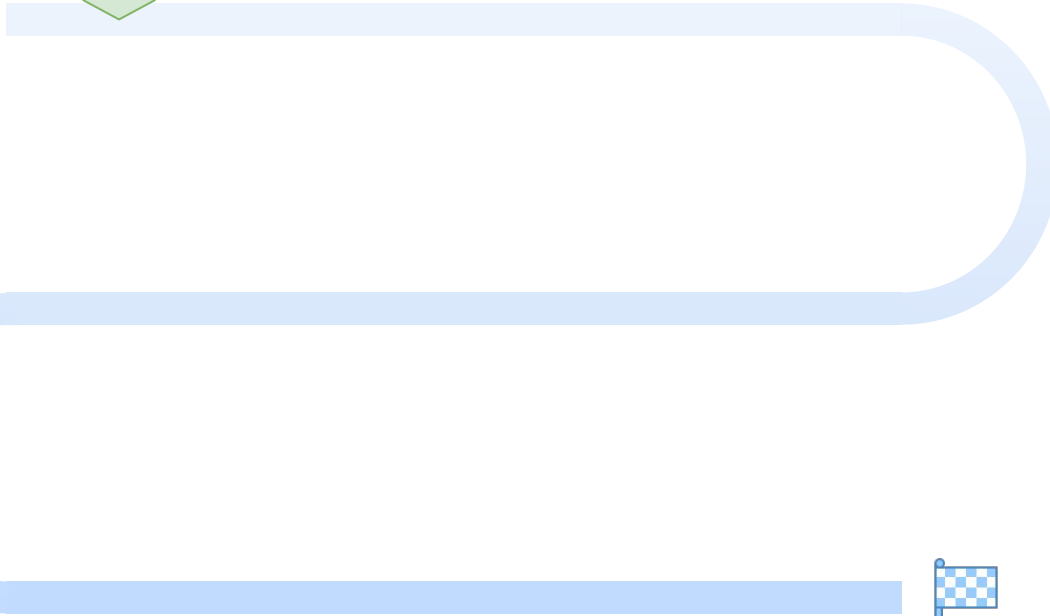


Python for DevOps





Python Fundamentals



Python Fundamentals

- ✓ **Environment setup** with `pyenv` and `venv`
- ✓ **Core data structures:** Lists, Tuples, Sets, Dictionaries
- ✓ **Control flow:** if/else and for/while Loops
- ✓ **Structuring code** with functions and classes
- ✓ **Variable arguments:** `*args` and `**kwargs`
- ✓ Among other topics

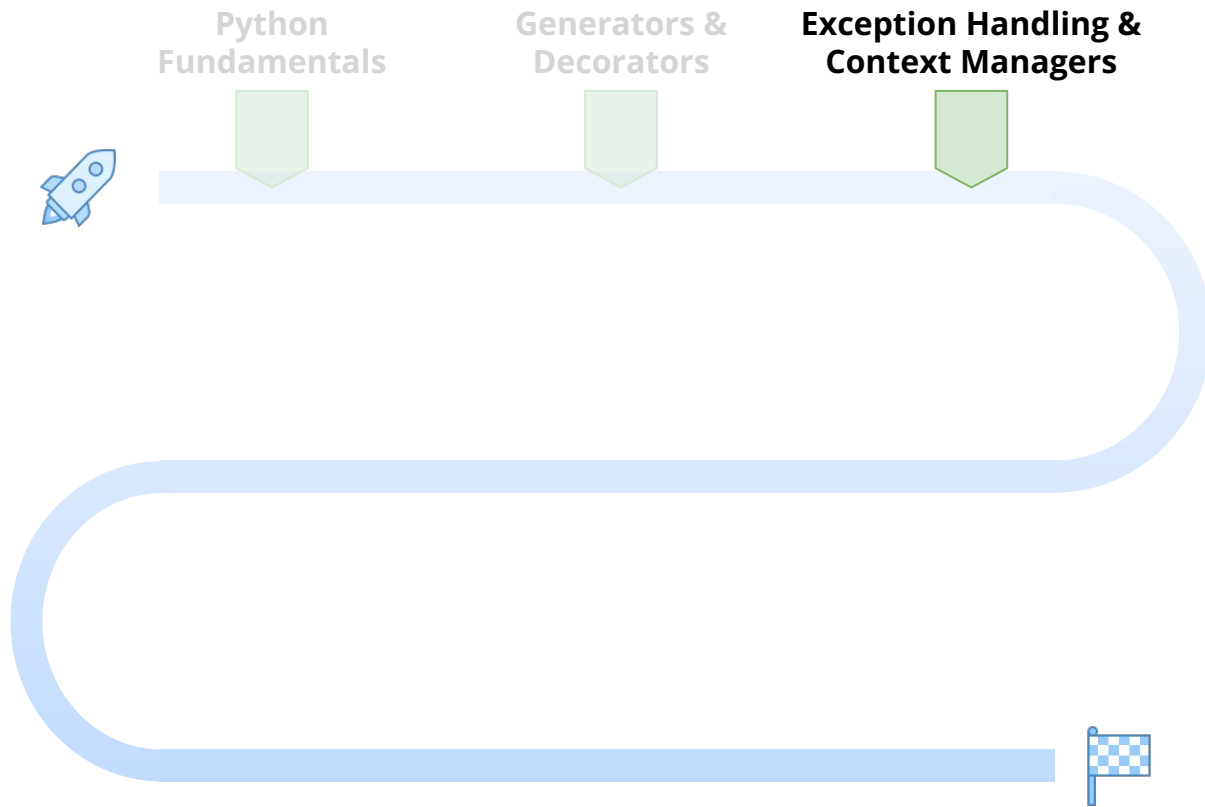
Python
Fundamentals

Generators &
Decorators



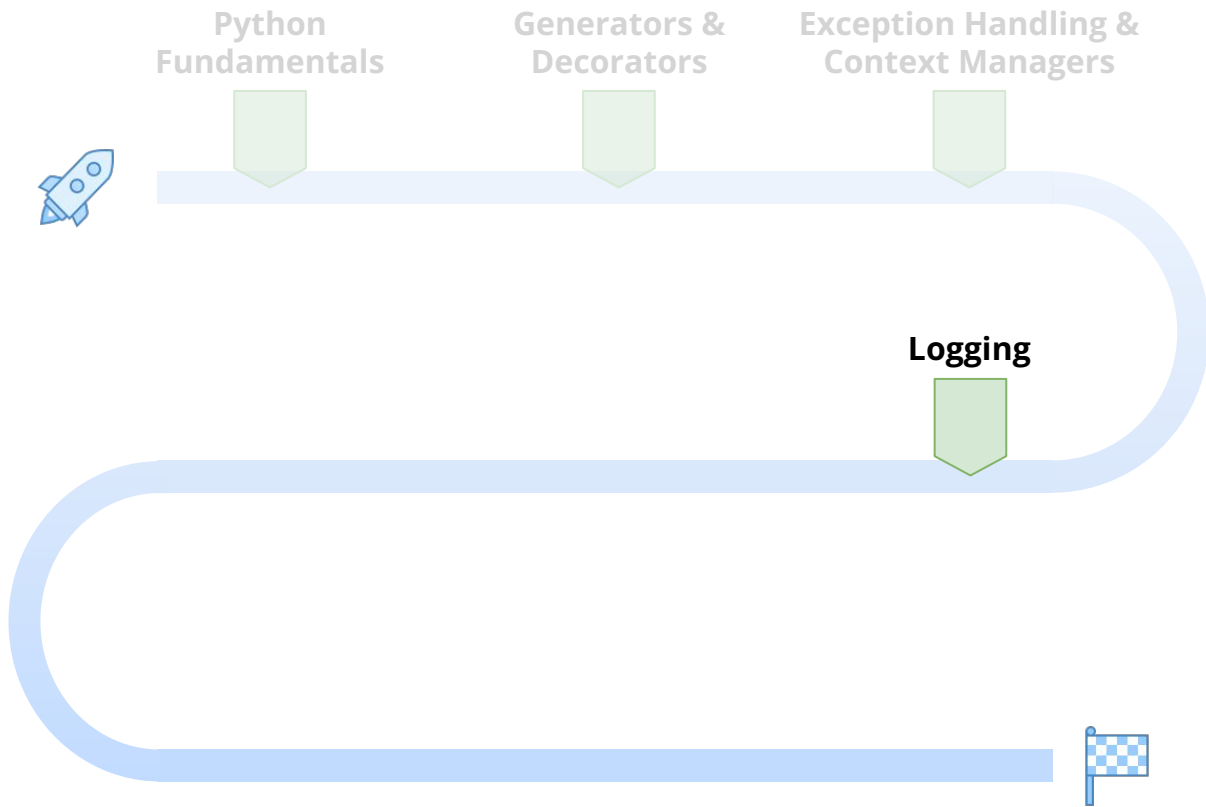
Generators and Decorators

- ✓ **Iteration protocol**
- ✓ **Generators** and the `yield` keyword
- ✓ **Lazy evaluation** and **memory-efficient** pipelines
- ✓ Functions as **first-class objects**
- ✓ **Decorators** and the `@` syntax
- ✓ **Decorator factories** and `functools.wraps`
- ✓ Among other topics



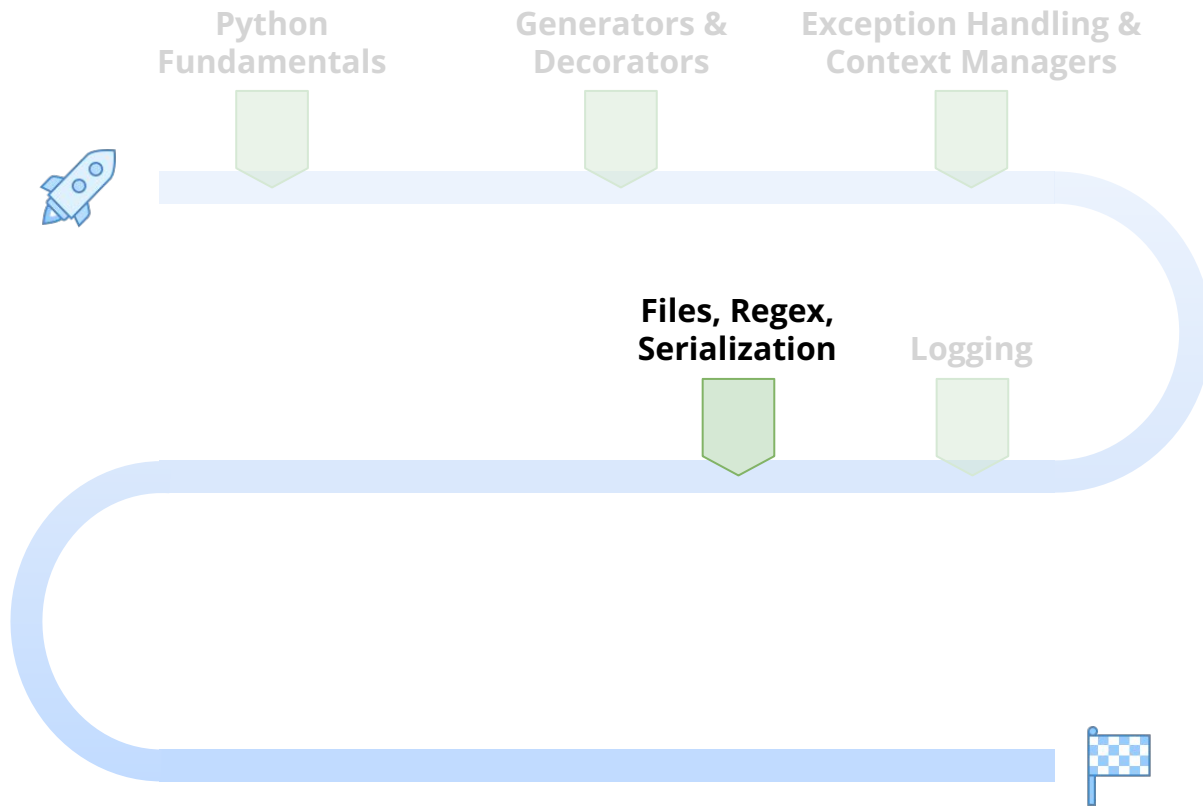
Exception & Context Management

- ✓ The `try/except/else/finally` block
- ✓ **Raising and handling exceptions**
- ✓ Common **built-in exceptions**
- ✓ Creating **custom exceptions**
- ✓ **Resource management** with the `with` statement
- ✓ Custom **context managers**
- ✓ Among other topics



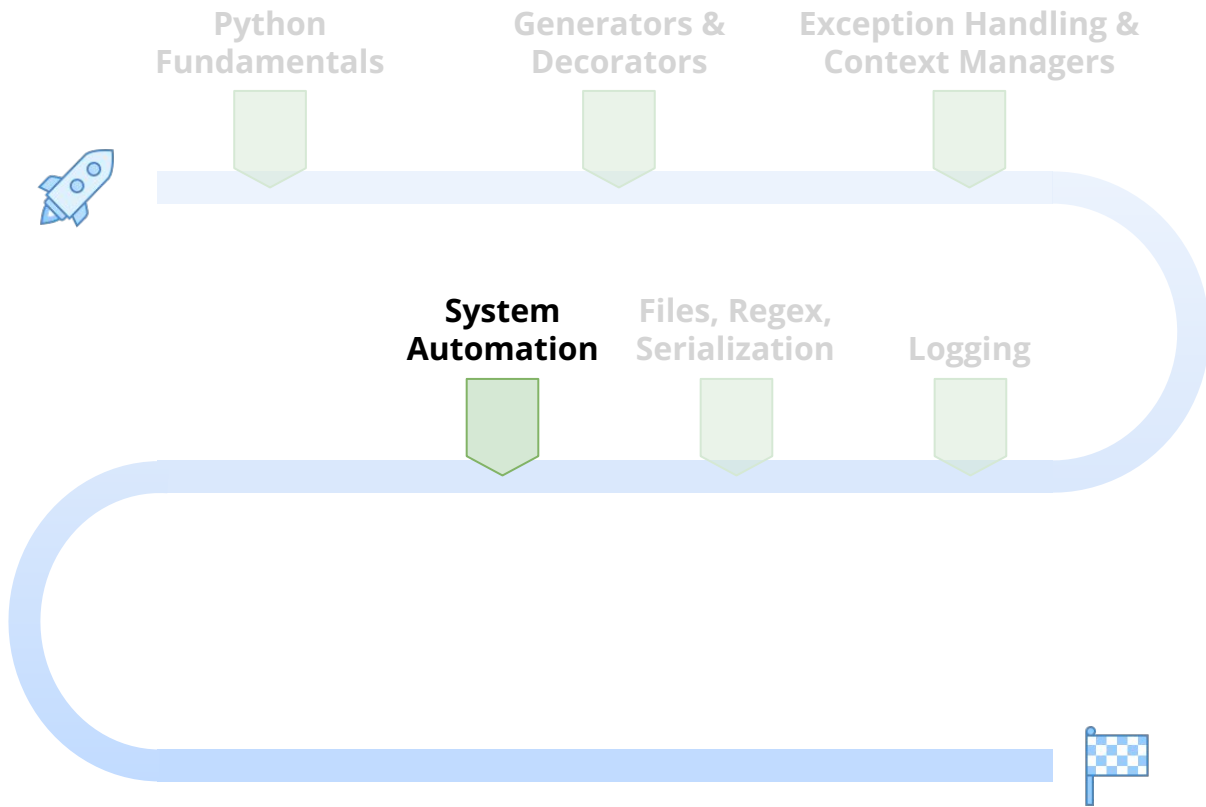
Production-Grade Logging

- ✓ **The `logging` module:** Loggers, Handlers, Formatters
- ✓ **File handlers** and **automated log rotation**
- ✓ **Structured logging** with JSON
- ✓ **Declarative** and **dynamic** logging configuration
- ✓ Among other topics



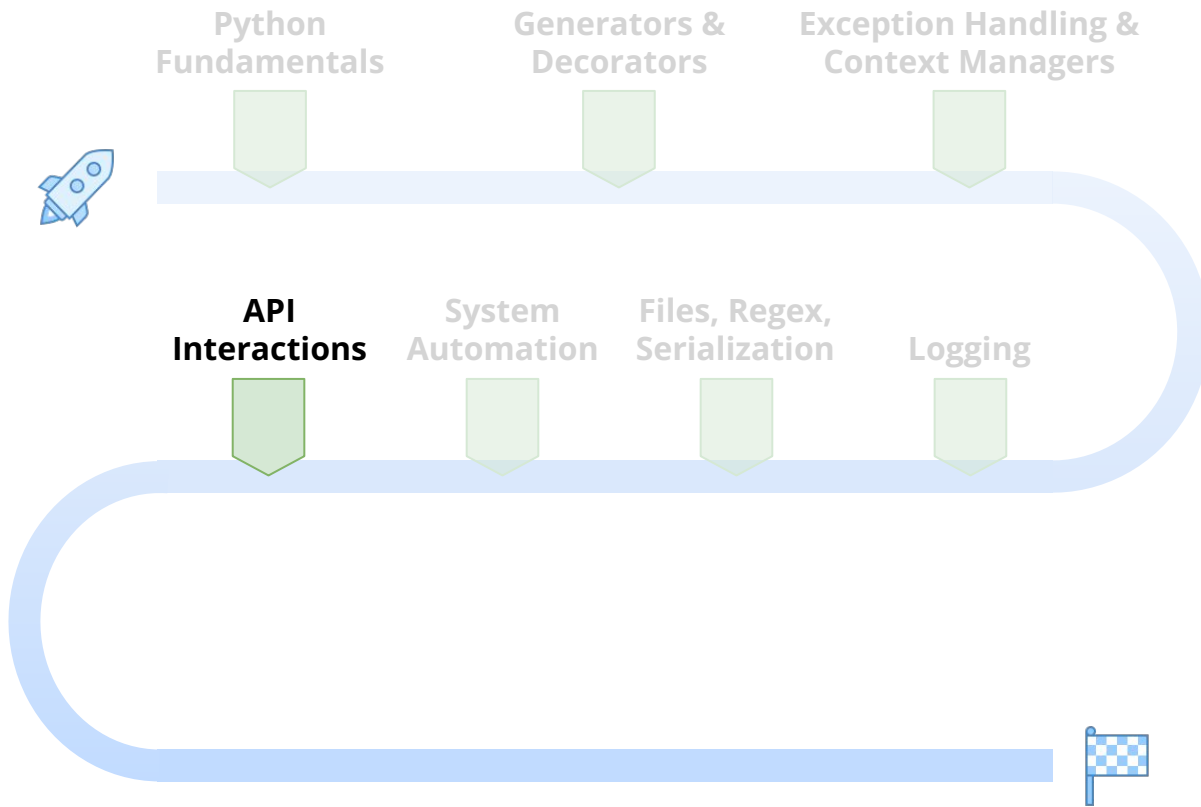
Files, Regex, Serialization

- ✓ **Modern file system I/O** with `pathlib`
- ✓ **Regular expressions (Regex)** for text processing
- ✓ **JSON serialization and deserialization**
- ✓ Parsing and generating **YAML** with PyYAML
- ✓ Reading and writing **CSV** Files



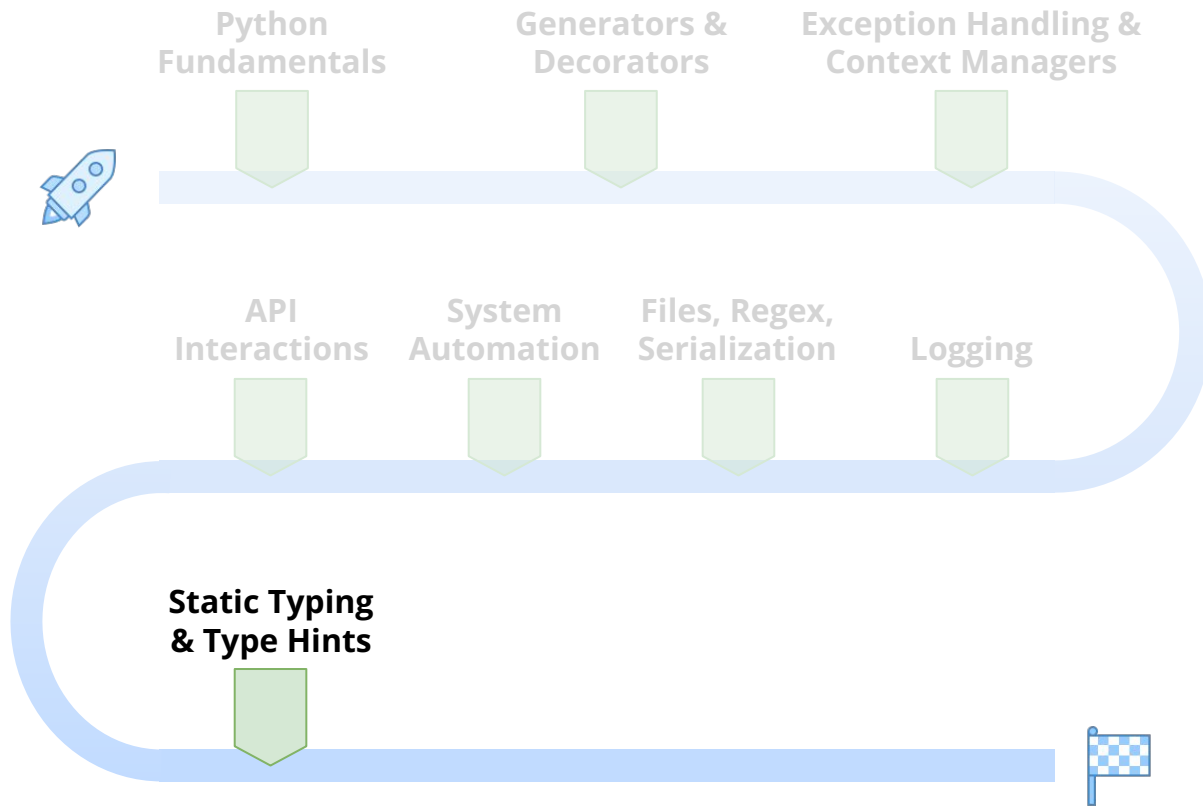
System Interaction & Automation

- ✓ Managing **environment variables** and `.env` files
- ✓ **File system operations** with `os` and `shutil`
- ✓ **Temporary files** and **directories**
- ✓ Running **external commands** with `subprocess.run`



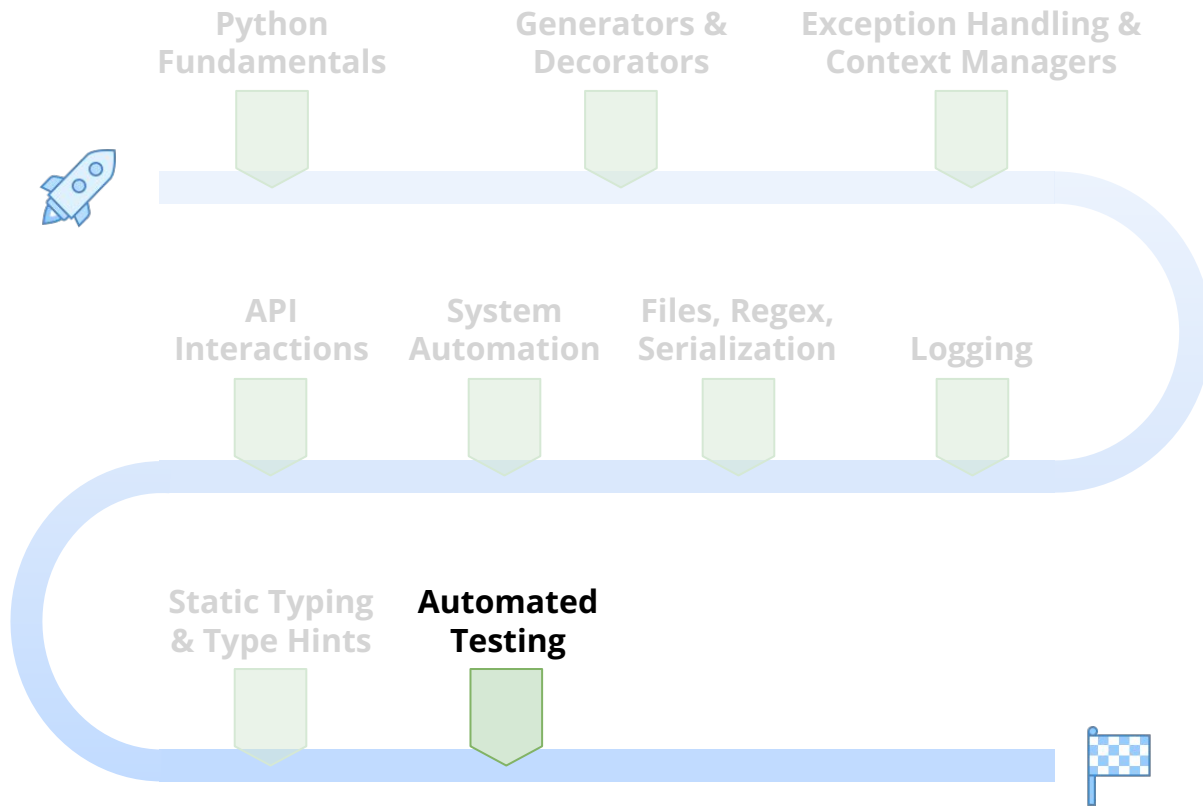
API Automation with **requests**

- ✓ Sending **HTTP requests**
- ✓ Handling **JSON payloads** and **query parameters**
- ✓ **API authentication:** Basic and Token-Based
- ✓ Handling **timeouts** and **errors**
- ✓ Implementing retries with **exponential backoff**



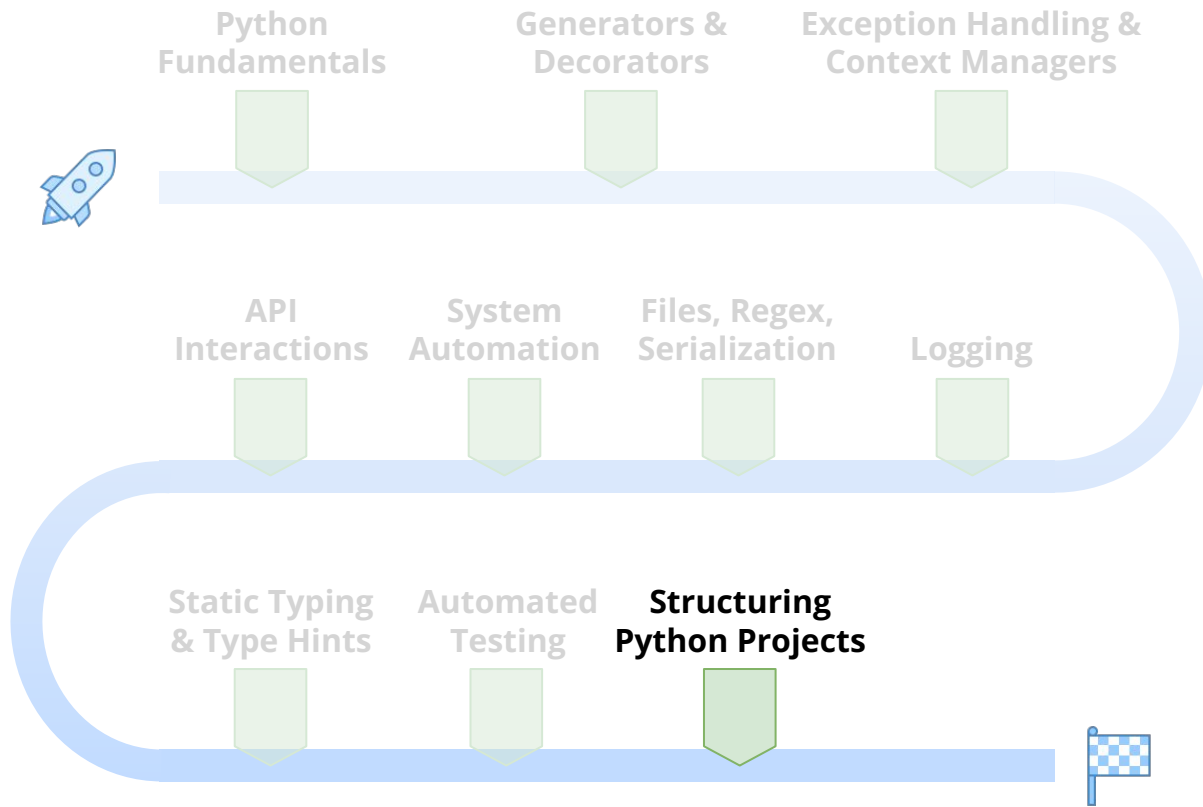
Static Typing & Type Hints

- ✓ **Type hint syntax** for functions and variables
- ✓ **Typing collections:** `list`, `dict`, `Union`, `Optional`
- ✓ **Defining data structures** with `TypedDict`
- ✓ Creating **generic functions** with `TypeVar`
- ✓ Defining **generic classes**
- ✓ **Typing decorators** with `ParamSpec`
- ✓ Among other topics



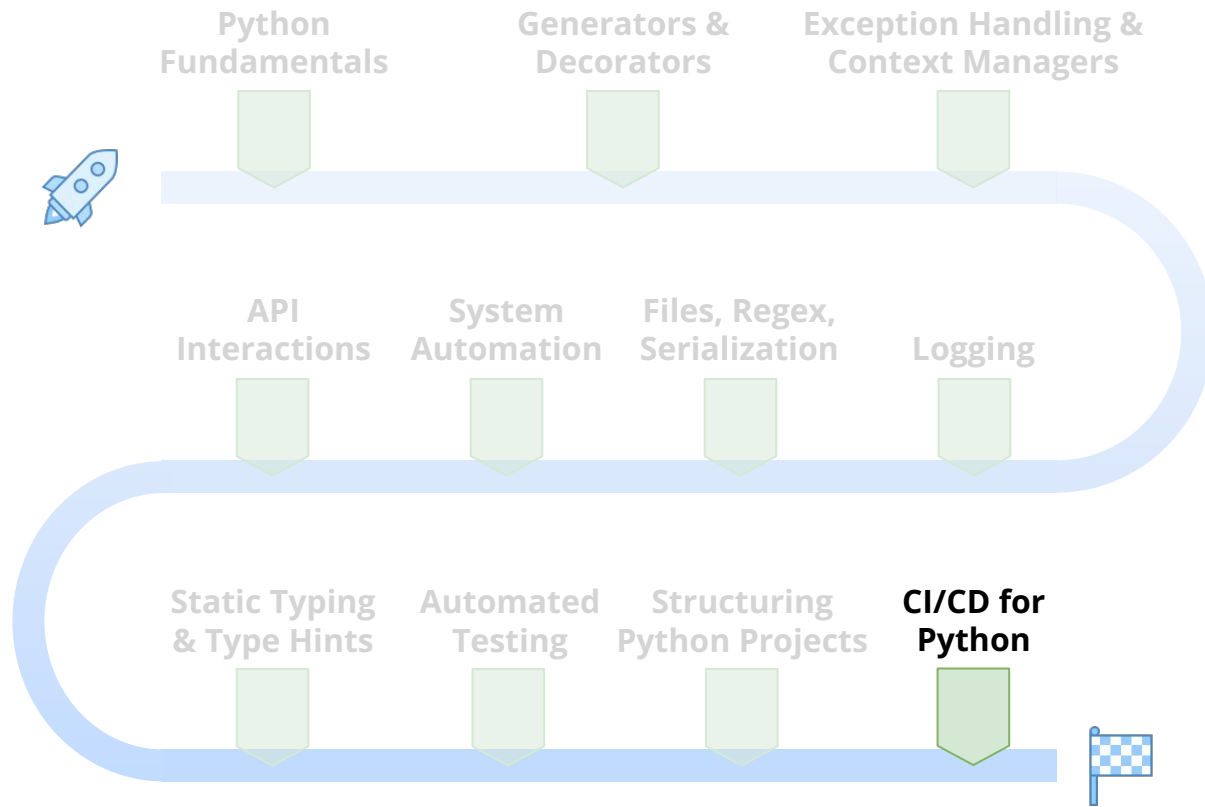
Automated Testing with **pytest**

- ✓ **Pytest assertions**
- ✓ **Fixtures** for setup & teardown
- ✓ **Parametrization** for data-driven tests
- ✓ **Controlling and categorizing test** runs with markers
- ✓ **Mocking** and **patching** with `unittest.mock`
- ✓ The **mock** fixture and **Mock** vs. **MagicMock**
- ✓ Among other topics



Structuring Python Projects

- ✓ **Modules, packages**, and `__init__.py`
- ✓ Absolute vs. relative **imports**
- ✓ **Running modules** with `python -m`
- ✓ **Defining projects** with `pyproject.toml`
- ✓ Creating **installable CLI** tools with **entry points**
- ✓ Among other topics



CI/CD for Python

- ✓ **Linting** and **formatting** checks
- ✓ **Static type checks** and **security scans**
- ✓ **Integrating** tools and automated tests into **GitHub Actions workflows**
- ✓ **Building projects** into installable files
- ✓ **Publishing packages** to **TestPyPI** and **PyPI**

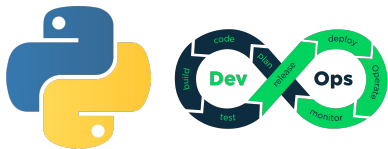
Introduction

Aligning Expectations



Aligning Expectations

What this course is about



Covering the most important aspects of working with Python in the context of DevOps



Practice what we discuss through quizzes and coding exercises



Going beyond the basics and covering topics like automated testing and mocking, generators, decorators, static typing, among many others!



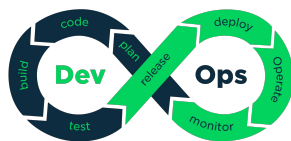
By the end of this course, you will have a strong **practical and theoretical foundation** of Python, as well as how to use many of its crucial features in the context of DevOps tasks, and you will be able to take this knowledge and apply it to **real-world, production projects**.

Aligning Expectations

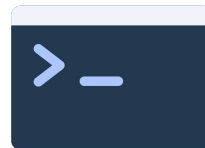
Requirements and prerequisites for making the most of this course



Familiarity (!) with programming:
programming logic, basic concepts such
as variables, loops, functions, among
other things.



**Familiarity with
DevOps concepts**



**Familiarity with the
terminal / command line**



**[Optional] Familiarity with
GitHub Actions**

Python for DevOps

Why Python for DevOps?



Why Python for DevOps?

Understand the benefits of using Python for automation and DevOps tasks

Readability and Simplicity

Python's clean syntax resembles plain English, lowering barriers and making scripts easier to write, read, and maintain, even for those who aren't full-time programmers.

Extensive Standard Library

Python comes with "batteries included." Modules like `os`, `sys`, `subprocess`, `json`, `csv`, among others, provide powerful tools for common DevOps tasks.

Vast Ecosystem

The Python Package Index (PyPI) hosts hundreds of thousands of third-party packages. Libraries like `requests`, `boto3`, `PyYAML`, `pytest` are indispensable for DevOps.

Cross-Platform

Python runs consistently on Linux, macOS, and Windows, making your automation scripts portable.

Integration

Python acts as an excellent "glue code," easily integrating with other languages, APIs, and existing command-line tools.

Community

A large, active community means abundant tutorials, forums, and pre-built solutions for common problems.

Python for DevOps

Setting Up Python



Setting Up Python

Section overview

1 Installing and configuring Python

1. Difference between system Python and CPython
2. Installing multiple Python versions with `pyenv`

2 Virtual Environments

1. Creating and managing virtual environments
2. Installing dependencies into virtual environments

3 Running Python code

1. Interactive coding with the REPL
2. Writing and running `.py` files
3. Using Jupyter Lab and Notebooks for advanced interactive coding

Python for DevOps

Installing and Configuring Python



Installing and Configuring Python

Understand different Python versions and best practices for managing them



System Python

Python interpreter that comes pre-installed with your operating system. Often required for system scripts and utilities. Treat it as part of the OS and avoid installing packages into it globally, especially using `sudo pip`.

CPython

This is the reference implementation of Python, written in C. When you download from python.org or use tools like Homebrew or `pyenv`, you typically get CPython. It's the most common and compatible version.

Installing from python.org

Likely the simplest method, recommended for Windows/macOS direct installations.

OS Package Managers

Using `apt`, `yum`, `brew`, installs a single version alongside system Python.

`pyenv` (recommended)

Ideal for managing multiple Python versions without touching system Python.

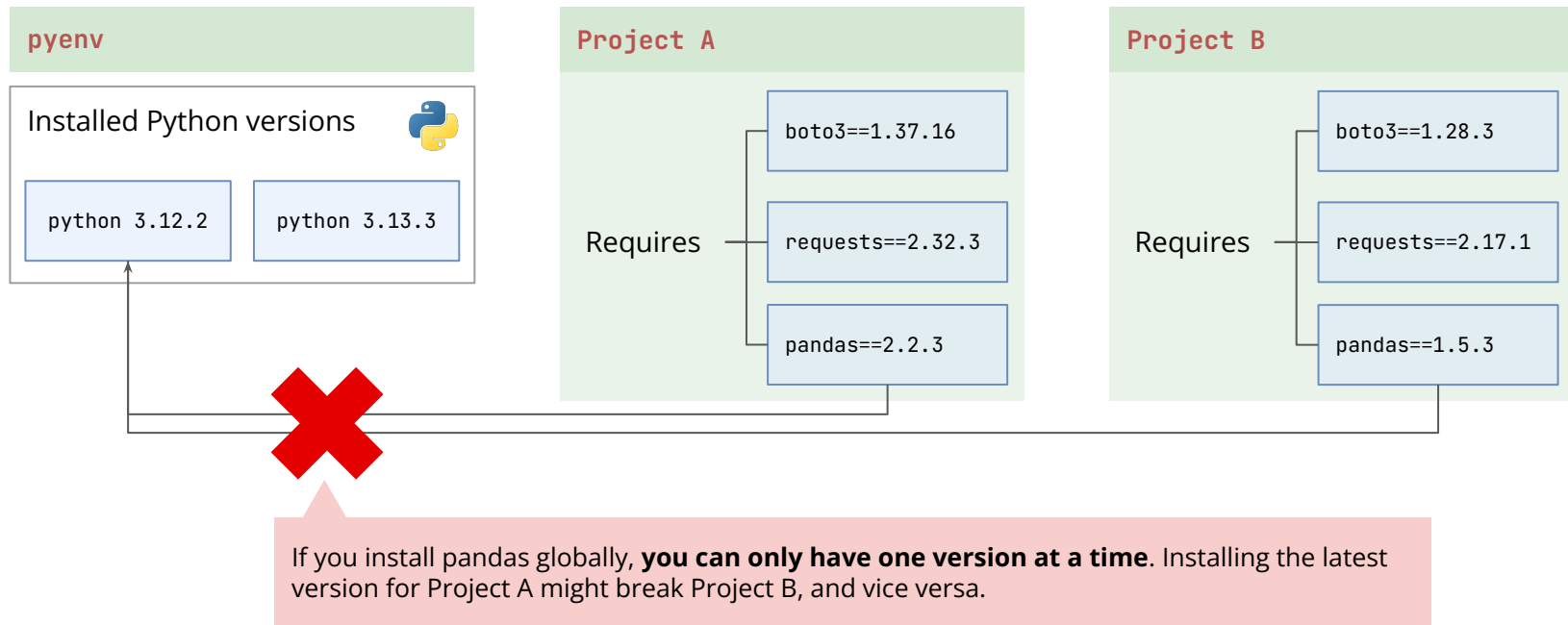
Python for DevOps

Virtual Environments



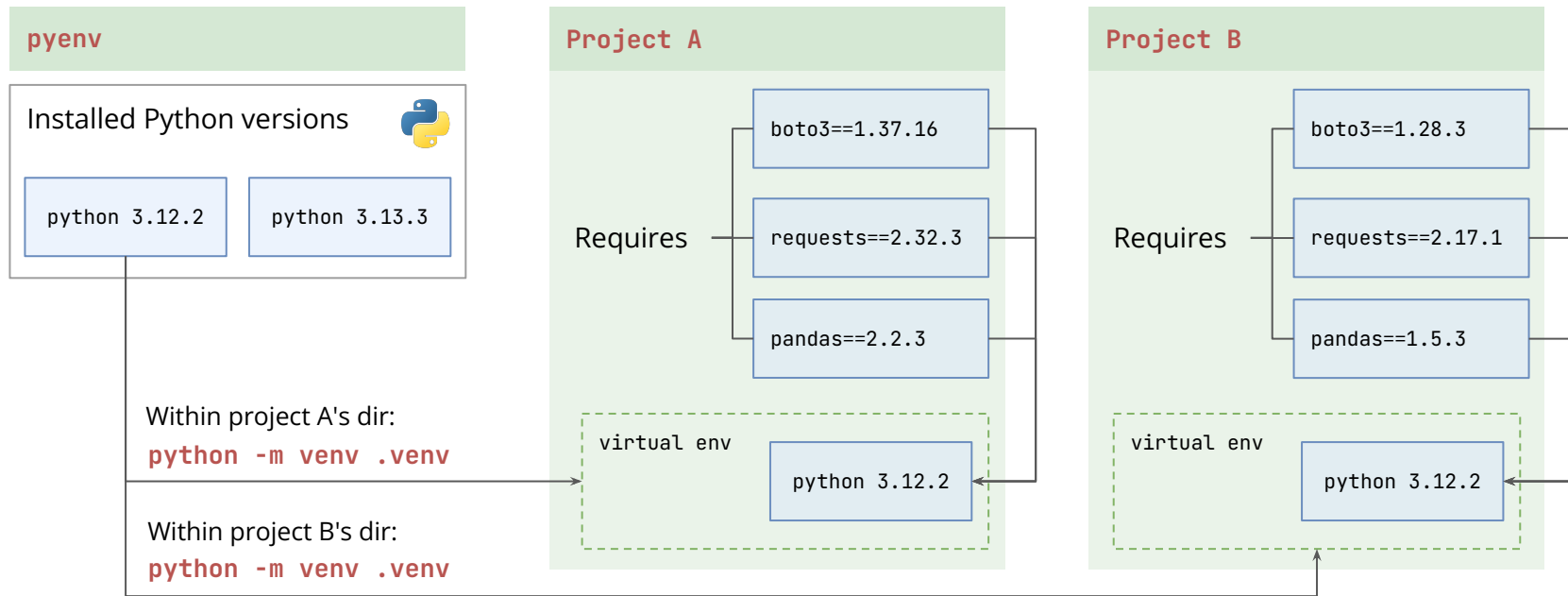
Virtual Environments

Understand how to isolate and manage project dependencies



Virtual Environments

Understand how to isolate and manage project dependencies



Python for DevOps

Python Fundamentals



Python Fundamentals

Section overview

1 Language Fundamentals

1. Variables and Data Types
2. Comments
3. Primitive types and string manipulation

2 Core Data Structures

1. Lists, tuples, and sets
2. Dictionaries

3 Control Flow

1. Conditional logic with `if`, `elif`, and `else`
2. Looping with `for` and `while`
3. Controlling loops with `break` and `continue`

Python Fundamentals

Section overview

4 Efficient Iteration and Data Creation

1. List, Set, and Dictionary comprehensions
2. Generating sequences with `range()`
3. Combining iterables with `enumerate()` and `zip()`

5 Structuring Code with Functions and Classes

1. Defining functions, parameters, and return values
2. Working with lambda functions
3. Variable Arguments with `*args` and `**kwargs`
4. Introduction to classes, methods, and inheritance

Python for DevOps

Lists, Tuples and Sets



Lists, Tuples and Sets

Understand the characteristics and use-cases for each of these collections

	Lists	Tuples	Sets
Characteristics	<pre>servers = ['web01', 'db01', 'web01']</pre> ✓ Ordered: respects the insertion order ✓ Mutable: items can be added, removed, or modified ✓ Allows duplicate items: equal numbers, strings, or objects can belong to the same list.	<pre>redis_cfg = ('10.0.5.1', 6379)</pre> ✓ Ordered: respects the insertion order ✓ Immutable: after creation, cannot be modified ✓ Allows duplicate items: equal numbers, strings, or objects can belong to the same list.	<pre>allowed_ports = {22, 80, 443}</pre> ✓ Unordered: contents may be returned out of order ✓ Mutable: items can be added, removed or modified ✓ Does not allow duplicates: equal items will be dropped.
Use-cases	• You need an editable, position-aware sequence. Some examples include: ◦ Ordered deployment steps ◦ Log lines or file reads ◦ Task queues	• Data is fixed and positional, with multiple ordered values representing a higher construct. Some examples include: ◦ <code>(host, port)</code> pairs ◦ RGB colors ◦ Multiple-return values: return <code>(stdout, stderr, rc)</code> and callers unpack it	• Focus is on deduplicating the data or performing membership checks. Some examples include: ◦ Deduplicating values in multiple occasions (e.g., IPs extracted from a series of access logs) ◦ Allowed-ports whitelist

Python for DevOps

Conditional Code Execution



Conditional Code Execution

Branch and control the execution of code via **if**, **else**, and **elif** constructs

```
if <condition>:  
    print('Condition is true.')  
print('Script continues...')
```

Simple **if** conditions will execute the nested code block only when the condition evaluates to a truthy value.

```
if <condition>:  
    print('Condition is true.')else:  
    print('Condition is false.')  
print('Script continues...')
```

if/else blocks will execute the nested code block under the if statement when the condition evaluates to a truthy value; if the condition evaluates to a falsy value, the nested code block under the else statement will be executed.

```
if <condition1>:  
    print('Condition1 is true.')elif <condition2>:  
    print('Condition1 is false.')    print('Condition2 is true.')else:  
    print('Condition1 is false.')    print('Condition2 is false.')  
print('Script continues...')
```

if/elif/else blocks provide a way to chain multiple if checks and execute a single block of code when any of the conditions evaluates to true. Conditions are evaluated top-to-bottom, so the first condition that evaluates to true will have its code block executed.

Conditional Code Execution

Branch and control the execution of code via **if**, **else**, and **elif** constructs

Falsy values

- The boolean **False**
- The special object **None**
- Zero of any numeric type: **0** (int), **0.0** (float)
- Empty sequences and collections:
 - a. Empty lists: **[]**
 - b. Empty tuples: **()**
 - c. Empty strings: **""**
 - d. Empty dictionaries: **{}**

Truthy values

- ✓ The boolean **True**
- ✓ Most other values, including non-empty sequences/collections (like **["server1"]** or **"error"**), non-zero numbers (**1**, **10.5**).

Python will automatically coerce different types to their boolean equivalent when used within **if** conditions. Therefore, simply mentioning falsy values will result in **False**, and simply mentioning truthy values will result in **True**.

```
users = ["user1", "user2"]

if users:
    print("Users have elements.")
else:
    print("Users is empty.")

config = {}

if config:
    print("Config has elements.")
else:
    print("Config is empty.")
```

Python for DevOps

Functions



Functions

An overview of the anatomy of a function in Python

A standard Python function

```
def reboot_server(hostname):  
    """Reboots the specified server.  
  
    Args:  
        hostname (str): The host to reboot.  
  
    Returns:  
        bool: True if successfully rebooted.  
    """  
    print(f"Rebooting {hostname}...")  
    # ...insert real reboot logic here...  
    return True
```

Functions

An overview of the anatomy of a function in Python

A standard Python function

```
def reboot_server(hostname):  
    """Reboots the specified server.  
  
    Args:  
        hostname (str): The host to reboot.  
  
    Returns:  
        bool: True if successfully rebooted.  
    """  
    print(f"Rebooting {hostname}...")  
    # ...insert real reboot logic here...  
    return True
```

Function declaration

Contains the keyword `def`, the function name, and an optional list of parameters between parenthesis.

Functions

An overview of the anatomy of a function in Python

A standard Python function

```
def reboot_server(hostname):  
    """Reboots the specified server.  
  
    Args:  
        hostname (str): The host to reboot.  
  
    Returns:  
        bool: True if successfully rebooted.  
    """  
    print(f"Rebooting {hostname}...")  
    # ...insert real reboot logic here...  
    return True
```

Function declaration

Contains the keyword `def`, the function name, and an optional list of parameters between parenthesis.

Docstring

The first statement inside the body. Must be between triple quotes. It explains purpose, params, return value, and is returned when calling `help(function_name)` and in IDE hints. It's best practice to list the arguments under the **Args** section and the return type under the **Returns** section.

Functions

An overview of the anatomy of a function in Python

A standard Python function

```
def reboot_server(hostname):  
    """Reboots the specified server.  
  
    Args:  
        hostname (str): The host to reboot.  
  
    Returns:  
        bool: True if successfully rebooted.  
    """  
  
    print(f"Rebooting {hostname}...")  
    # ...insert real reboot logic here...  
    return True
```

Function declaration

Contains the keyword `def`, the function name, and an optional list of parameters between parenthesis.

Docstring

The first statement inside the body. Must be between triple quotes. It explains purpose, params, return value, and is returned when calling `help(function_name)` and in IDE hints. It's best practice to list the arguments under the **Args** section and the return type under the **Returns** section.

Indented function body

The code that runs when the function is called.

Functions

An overview of the anatomy of a function in Python

A standard Python function

```
def reboot_server(hostname):  
    """Reboots the specified server.  
  
    Args:  
        hostname (str): The host to reboot.  
  
    Returns:  
        bool: True if successfully rebooted.  
        None  
  
    print(f"Rebooting {hostname}...")  
    # ...insert real reboot logic here...  
    return True
```

Function declaration

Contains the keyword **def**, the function name, and an optional list of parameters between parenthesis.

Docstring

The first statement inside the body. Must be between triple quotes. It explains purpose, params, return value, and is returned when calling **help(function_name)** and in IDE hints. It's best practice to list the arguments under the **Args** section and the return type under the **Returns** section.

Indented function body

The code that runs when the function is called.

Optional return statement

Sends a result back to the caller; omitting it returns **None**.

Functions

An overview of the anatomy of a function in Python

A standard Python function

```
def reboot_server(hostname):  
    """Reboots the specified server.  
  
    Args:  
        hostname (str): The host to reboot.  
  
    Returns:  
        bool: True if successfully rebooted.  
    """  
    print(f"Rebooting {hostname}...")  
    # ...insert real reboot logic here...  
    return True
```

Function declaration

Contains the keyword `def`, the function name, and an optional list of parameters between parenthesis.

Docstring

The first statement inside the body. Must be between triple quotes. It explains purpose, params, return value, and is returned when calling `help(function_name)` and in IDE hints. It's best practice to list the arguments under the **Args** section and the return type under the **Returns** section.

Indented function body

The code that runs when the function is called.

Optional return statement

Sends a result back to the caller; omitting it returns `None`.

Python for DevOps

Generators and Decorators



Generators and Decorators

Section overview

1 Generators and lazy evaluation

1. Generator functions and the `yield` keyword
2. Understanding generator state and exhaustion
3. Comparing eager vs. lazy evaluation (`return` vs. `yield`)
4. Building memory-efficient data pipelines

2 Functions as first-class objects

1. Assigning functions to variables
2. Passing functions as arguments to other functions
3. Returning functions from functions (factory pattern)
4. Storing functions in data structures

Generators and Decorators

Section overview

3

Decorators

1. Introduction to decorator syntax (@)
2. Passing arguments to decorators (decorator factories)
3. Handling return values and exceptions in wrappers
4. Preserving function metadata with `@functools.wraps`
5. Applying multiple decorators (stacking)

Python for DevOps

The Iteration Protocol

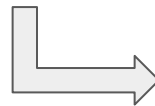


The Iteration Protocol

We use `for item in sequence:` all the time. But how does Python get each `item`?

- **Iterable:** An object that can be looped over. It's anything you can put on the right side of the `in` keyword in a `for` loop. Examples include lists, tuples, strings, dictionaries, sets, files, and range objects.
 - An object is considered **iterable** if it implements the `__iter__()` special method.
 - The `__iter__()` method returns an **iterator**.
- **Iterator:** An object that produces the *next* value in a sequence when asked. It "remembers" its position in the sequence.
 - An object is an **iterator** if it implements the `__next__()` special method. When there are no more items, `__next__()` raises the `StopIteration` exception.
 - Iterators normally also implement the `__iter__()` method, which makes them iterables too.

```
class CountTo:  
    def __init__(self, max_value):  
        self.max = max_value  
  
    def __iter__(self):  
        # Each new for-loop call  
        # gets its own iterator  
        return CountToIter(self.max)
```



- ✓ **Supports nested loops:** Decoupling iterables from iterators allow us to instantiate a single iterable and use it in nested loops without consuming the values from a single iterator.

```
class CountToIter:  
    def __init__(self, max_value):  
        self.max = max_value  
        self.curr = 1  
  
    def __iter__(self):  
        # Iterators are iterable  
        return self  
  
    def __next__(self):  
        if self.curr <= self.max:  
            val = self.curr  
            self.curr += 1  
            return val  
        else:  
            raise StopIteration
```

Python for DevOps

Error Handling and Context Managers



Error Handling and Context Managers

Section overview

1 Exception handling fundamentals

1. The `try`, `except`, `else`, and `finally` blocks
2. Handling multiple and specific exception types
3. Design pattern: "easier to ask for forgiveness than permission"

2 Working with built-in exceptions

1. Common exceptions: `OSError`, `KeyError`, `IndexError`
2. Differentiating `ValueError` and `TypeError`
3. Handling `AttributeError` and `ImportError`

3 Raising and creating exceptions

1. The `raise` statement
2. Creating custom exception classes
3. Adding context to custom exceptions with `__init__`

Error Handling and Context Managers

Section overview

- 4 Context managers for resource management
 1. Manual cleanup with `try...finally`
 2. Automatic cleanup with the `with` statement
 3. Creating custom context managers with classes (`__enter__`, `__exit__`)
 4. Creating custom context managers with the `@contextlib.contextmanager` decorator

Python for DevOps

Exceptions in Python



Exceptions in Python

Handle errors and design more robust code

try-except-finally block

```
numerator = 1
denominator = 0
try:
    # Try executing operations that may fail
    result = numerator / denominator
    print(f"Result: {result}")
except ZeroDivisionError:
    print("I will handle a ZeroDivisionError.")
except Exception as e:
    print("I will handle any other exception.")
else:
    print("I will run if no exceptions occur.")
finally:
    print("I will always run.")
```

Exceptions in Python

Handle errors and design more robust code

try-except-finally block

```
numerator = 1
denominator = 0
try:
    # Try executing operations that may fail
    result = numerator / denominator
    print(f"Result: {result}")
except ZeroDivisionError:
    print("I will handle a ZeroDivisionError.")
except Exception as e:
    print("I will handle any other exception.")
else:
    print("I will run if no exceptions occur.")
finally:
    print("I will always run.")
```

The **try** block

Contains code that might raise an exception.

Exceptions in Python

Handle errors and design more robust code

try-except-finally block

```
numerator = 1
denominator = 0
try:
    # Try executing operations that may fail
    result = numerator / denominator
    print(f"Result: {result}")
except ZeroDivisionError:
    print("I will handle a ZeroDivisionError.")
except Exception as e:
    print("I will handle any other exception.")
else:
    print("I will run if no exceptions occur.")
finally:
    print("I will always run.")
```

The **try** block

Contains code that might raise an exception.

The **except** block

Allows handling individual exceptions. It's best practice to be specific about the exceptions you catch. Don't use a generic **except Exception:** unless absolutely necessary, as it can hide unexpected bugs.

It's possible to stack multiple **except** blocks, so that different exceptions are handled.

It's possible to catch multiple types: **except (FileNotFoundError, PermissionError)**

It's possible to access the exception object with: **except Exception as e:**

Exceptions in Python

Handle errors and design more robust code

try-except-finally block

```
numerator = 1
denominator = 0
try:
    # Try executing operations that may fail
    result = numerator / denominator
    print(f"Result: {result}")
except ZeroDivisionError:
    print("I will handle a ZeroDivisionError.")
except Exception as e:
    print("I will handle any other exception.")
else:
    print("I will run if no exceptions occur.")
finally:
    print("I will always run.")
```

The **else** block

Contains code that will be executed if and only if the code within the **try** block executes successfully.

Exceptions in Python

Handle errors and design more robust code

try-except-finally block

```
numerator = 1
denominator = 0
try:
    # Try executing operations that may fail
    result = numerator / denominator
    print(f"Result: {result}")
except ZeroDivisionError:
    print("I will handle a ZeroDivisionError.")
except Exception as e:
    print("I will handle any other exception.")
else:
    print("I will run if no exceptions occur.")
finally:
    print("I will always run.")
```

The **else** block

Contains code that will be executed if and only if the code within the **try** block executes successfully.

The **finally** block

Contains code that will be always executed, independently of whether exceptions were raised. Recommended place for cleanup logic.

Python for DevOps

Thinking in Exceptions



Thinking in Exceptions

Leverage exceptions to restructure and improve code design

Look Before You Leap (LBYL) Approach

```
import os
filepath = "/path/to/maybe/file.txt"
if os.path.exists(filepath):
    if os.access(filepath, os.R_OK):
        with open(filepath, 'r') as f:
            # ... process file ...
    else:
        print("Error: No read permission")
else:
    print("Error: File does not exist")
```

Easier to Ask for Forgiveness than Permission (EAFP)

```
import os
filepath = "/path/to/maybe/file.txt"
try:
    with open(filepath, 'r') as f:
        # ... process file ...
except FileNotFoundError:
    print(f"Error: File not found.")
except PermissionError:
    print(f"Error: Permission denied.")
except Exception as e:
    print(f"An unexpected error occurred.")
```

Why EAFP over LBYL?

- ✓ **Cleaner Code:** Separates the main "happy path" logic from the error-handling logic, improving readability.
- ✓ **Avoids Race Conditions:** LBYL checks can be invalidated between the check and the action (file deleted after `os.path.exists` but before `open`). EAFP performs the action and handles errors atomically.
- ✓ **Handles Specific Errors:** `try ... except` allows you to catch and respond differently to **specific types** of exceptions (`FileNotFoundError` vs. `PermissionError`).
- ✓ **Promotes Robustness:** Encourages thinking about **what could go wrong** during an operation and how to recover, rather than trying to predict every possible state beforehand. Letting unexpected errors crash (fail fast) signals a bug or situation needing developer attention.

Python for DevOps

The Context Manager Protocol



The Context Manager Protocol

We use the built-in `with ...:` all the time. But how can we build our own?

- Classes that implement the `__enter__(self)` and `__exit__(self, exc_type, exc_val, exc_tb)` methods can be used inside the `with` expression.
- `__enter__` is used for setup, and its returned value will be bound to the `as` target.
- `__exit__` will always run when the indented block under `with` finishes, independently of whether an exception was raised or not. If it returns `True`, it suppresses exceptions; otherwise, the raised exception will propagate.
- Exception information is passed to the `__exit__` method through positional parameters.
- Custom classes can implement these methods for tailored resource management.

```
class DummyCM:
    def __enter__(self):
        print("Entering")
        return "resource"

    def __exit__(self, exc_type, exc_val, exc_tb):
        print(f"Exiting; exception={exc_type}")
        # Returning False re-raises any exception
        return False

try:
    with DummyCM() as res:
        print("Inside with, got:", res)
        raise RuntimeError("Test error")
except Exception as e:
    print("Caught outside:", e)
```

```
Entering
Inside with, got: resource
Exiting; exception=<class 'RuntimeError'>
Caught outside: Test error
```



Python for DevOps

Logging for Operations



Logging for Operations

Section overview

1 Logging fundamentals

1. Why logging is essential for DevOps
2. The anatomy of the logging module: loggers, handlers, formatters, and filters
3. Understanding log levels (`DEBUG`, `INFO`, `WARNING`, `ERROR`, `CRITICAL`)
4. Two-stage filtering: logger-level and handler-level

2 Logging to files

1. Basic `FileHandler` for writing to files
2. `RotatingFileHandler` for size-based log rotation
3. `TimedRotatingFileHandler` for time-based log rotation

Logging for Operations

Section overview

3 Structured logging with JSON

1. Configuring the `python-json-logger` formatter
2. Adding extra contextual fields to JSON logs
3. Logging exceptions with tracebacks as JSON

4 Declarative configuration

1. Configuring logging with `.ini` files (`fileConfig`)
2. Configuring logging with dictionaries (`dictConfig`)
3. Loading configuration from JSON files
4. Building and modifying logging configuration dynamically

Python for DevOps

Introduction to Logging



Introduction to Logging

Understand the foundations of logging and related good practices

- Act of recording events, messages, and data about the execution of your application or system. These records create a history of what happened, when it happened, and potentially why.
- Why logging is essential in DevOps?

Troubleshooting and Debugging

When something goes wrong (an error, a crash, unexpected behavior), logs are one of the main sources for understanding the sequence of events leading up to the failure.

Auditing and Security

Logs can provide an audit trail of significant actions (user logins, configuration changes, deployment triggers). This is vital for auditing, security analysis, and compliance requirements.

Monitoring and Alerting

Logs can act as a basis for metrics, trends, and alerts based on patterns or thresholds. This provides real-time insight into system health.

Understanding System Behavior

Logs provide visibility into how different components of a distributed system are interacting, which requests are being processed, and the general flow of execution, even when things are working correctly.

Introduction to Logging

Logging vs `print()`

- While `print()` is fine for quick, temporary debugging during development, it's insufficient for robust application logging!

Introduction to Logging

Logging vs `print()`

- While `print()` is fine for quick, temporary debugging during development, it's insufficient for robust application logging!

Lack of Levels

All `prints` look the same. Logging uses levels (`DEBUG`, `INFO`, `WARNING`, `ERROR`, `CRITICAL`) to categorize message severity, allowing filtering. Think about sifting through thousands of print messages versus filtering for just `ERROR` logs!

Introduction to Logging

Logging vs `print()`

- While `print()` is fine for quick, temporary debugging during development, it's insufficient for robust application logging!

Lack of Levels

All `prints` look the same. Logging uses levels (`DEBUG`, `INFO`, `WARNING`, `ERROR`, `CRITICAL`) to categorize message severity, allowing filtering. Think about sifting through thousands of print messages versus filtering for just `ERROR` logs!

Missing Context

Logs automatically add valuable context: timestamp, source file, function name, process ID. Adding this manually to `print` is tedious and inconsistent.

Introduction to Logging

Logging vs `print()`

- While `print()` is fine for quick, temporary debugging during development, it's insufficient for robust application logging!

Lack of Levels

All `prints` look the same. Logging uses levels (`DEBUG`, `INFO`, `WARNING`, `ERROR`, `CRITICAL`) to categorize message severity, allowing filtering. Think about sifting through thousands of print messages versus filtering for just `ERROR` logs!

Missing Context

Logs automatically add valuable context: timestamp, source file, function name, process ID. Adding this manually to `print` is tedious and inconsistent.

Inflexible Output

`print` goes to stdout. Logging can easily direct output to files, rotate logs, send over network, and handle multiple destinations without code changes.

Introduction to Logging

Logging vs `print()`

- While `print()` is fine for quick, temporary debugging during development, it's insufficient for robust application logging!

Lack of Levels

All `prints` look the same. Logging uses levels (`DEBUG`, `INFO`, `WARNING`, `ERROR`, `CRITICAL`) to categorize message severity, allowing filtering. Think about sifting through thousands of print messages versus filtering for just `ERROR` logs!

Missing Context

Logs automatically add valuable context: timestamp, source file, function name, process ID. Adding this manually to `print` is tedious and inconsistent.

Inflexible Output

`print` goes to stdout. Logging can easily direct output to files, rotate logs, send over network, and handle multiple destinations without code changes.

No structure

`print` outputs unstructured text. Logging supports structured formats (like JSON) easily consumed by analysis tools (Logstash, Splunk, DataDog, etc.).

Introduction to Logging

Think ahead and log relevant information

- **Anticipate Failures:**

- Ask yourself: "What information is needed if this function fails?"
- Log relevant inputs or state *before* potential failure points.

- **Record Milestones:** Log key events or state changes ("Service started", "Configuration loaded", "Deployment initiated").

- **Use Levels Wisely:**

- **DEBUG:** Detailed info for diagnosing problems (might be turned off in production).
- **INFO:** Confirmation of normal operation, significant events.
- **WARNING:** Indicate potential issues or unexpected situations that don't stop execution yet.
- **ERROR:** Problems preventing a specific operation from completing.
- **CRITICAL:** Severe errors likely causing application termination.

- **Structure for Machines:** Consider how logs will be parsed later (consistent formatting, key-value pairs, JSON).

Python for DevOps

Logging Anatomy



Logging Anatomy

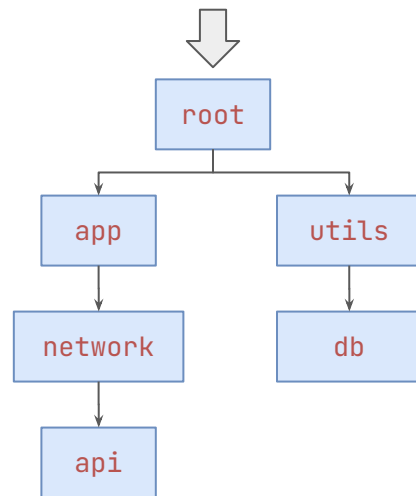
Understand the different components involved in logging in Python

- `logging.getLogger(<logger-name>)` retrieves a specific logger by name. A `logger` is an entrypoint that can be used by our scripts to send logs.
 - `<logger-name>` can use dots to define hierarchy. Loggers form a tree structure based on names. `myApp.network` is a child of `myApp`, which is a child of `root`.

```
logging.getLogger("app.network")
```

```
logging.getLogger("app.network.api")
```

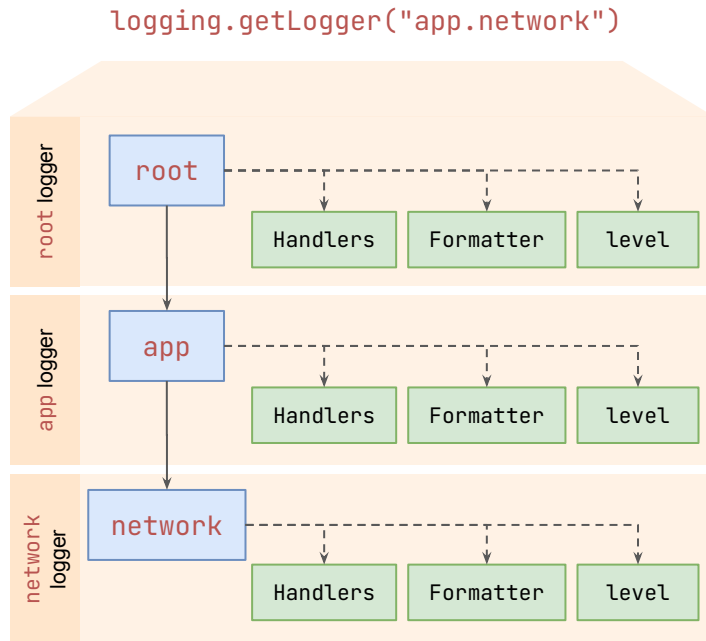
```
logging.getLogger("utils.db")
```



Logging Anatomy

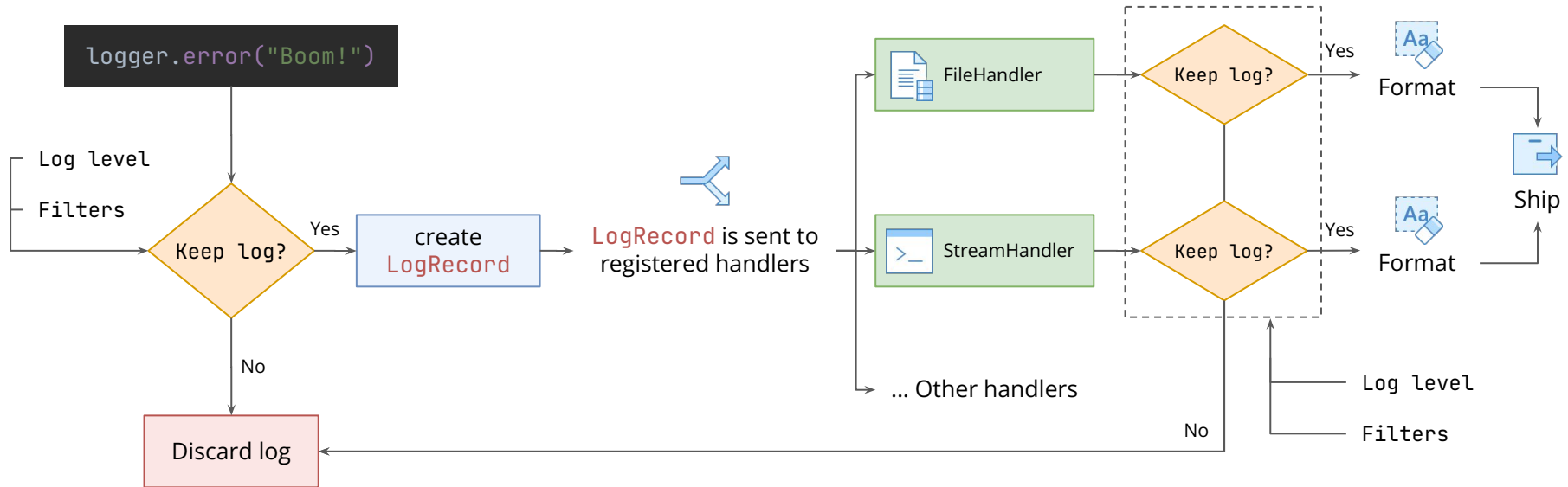
Understand the different components involved in logging in Python

- `logging.getLogger(<logger-name>)` retrieves a specific logger by name. A **logger** is an entrypoint that can be used by our scripts to send logs.
 - `<logger-name>` can use dots to define hierarchy. Loggers form a tree structure based on names. `myApp.network` is a child of `myApp`, which is a child of `root`.
- Loggers can have one or more **handlers**.
 - Handlers are responsible for formatting the logs and shipping them to their final destination.
 - We can have multiple handlers that process the same log message but deliver it in different formats to different destinations.
 - **Formatters** are attached to handlers.
- Both loggers and handlers retain or discard log messages based on level or more advanced **Filters**.
- By default (`logger.propagate = True`), messages processed by a logger are also passed to its parent's handlers.



Logging Anatomy

Understand the different components involved in logging in Python



Python for DevOps

Files, Regex, and Serialization



Files, Regex, and Serialization

Section overview

1 Modern file system operations

1. Introduction to the `pathlib` module and the `Path` object
2. Listing directory contents and traversing directories
3. Reading and writing files with `Path` objects

2 Regular expressions (regex)

1. Regex essentials: metacharacters, character classes, and special sequences
2. Quantifiers: greedy vs. non-greedy matching
3. Extracting data with capturing groups
4. Searching (`findall`), splitting (`split`), and replacing (`sub`) text

3 Handling JSON data

1. Serializing and deserializing JSON

Files, Regex, and Serialization

Section overview

4 Handling YAML data

1. Introduction to YAML syntax and the PyYAML library
2. Serializing and deserializing YAML data

5 Handling CSV data

1. Reading CSV files with `csv.reader` and `csv.DictReader`
2. Writing data to CSV files with `csv.writer` and `csv.DictWriter`

Python for DevOps

System Interaction and Automation



System Interaction and Automation

Section overview

1 Managing environment variables

1. Accessing, setting, and deleting environment variables with `os.environ`
2. Loading variables from `.env` files for local development

2 File system management with `os` and `shutil`

1. Listing directory contents
2. Creating and deleting files and directories
3. Copying and moving files and directories

3 Working with temporary files and directories

1. Using `tempfile` for secure, temporary storage
2. Creating `TemporaryFile`, `NamedTemporaryFile`, and `TemporaryDirectory`

System Interaction and Automation

Section overview

- 4 Running external commands with `subprocess`
 - 1. Introduction to `subprocess.run`
 - 2. Capturing standard output and standard error
 - 3. Handling subprocess errors (`CalledProcessError`)
 - 4. Managing timeouts (`TimeoutExpired`)

Python for DevOps

Interacting with APIs using requests



Interacting with APIs using requests

Section overview

1 Making HTTP requests

1. Sending **GET** requests to retrieve data
2. Passing query parameters for filtering and pagination
3. Sending **POST** requests with JSON payloads to modify data

2 Handling responses and errors

1. Checking HTTP status codes (`response.status_code`, `response.ok`)
2. Automatically raising exceptions for errors (`response.raise_for_status()`)

3 Authentication

1. Basic authentication (username/password)
2. Token-based authentication (API keys, bearer tokens)

Interacting with APIs using requests

Section overview

4 Building resilient API clients

1. Handling connection and read timeouts
2. Implementing simple retry logic for transient errors
3. Using exponential backoff and jitter for robust retries

Python for DevOps

Static Typing and Type Hints



Static Typing and Type Hints

Section overview

1 Typing fundamentals

1. Why use type hints in a dynamically typed language
2. Basic syntax for variables and functions
3. Common pitfalls and runtime behavior

2 Typing common collections

1. Typing lists, tuples, and sets
2. Typing dictionaries with key and value types
3. Handling multiple types with `Union` and `Optional`
4. Disabling type checks with `Any`
5. Structuring data with `TypedDict`

Static Typing and Type Hints

Section overview

3 Typing custom classes

1. Annotating methods and attributes
2. Handling forward references within a class

4 Creating generic functions and classes

1. Introduction to `TypeVar` for flexible functions
2. Using constrained and bounded `TypeVars`
3. Building generic classes that work with multiple types

5 Typing advanced patterns

1. Typing decorators
2. Typing generators and iterables

Python for DevOps

Automated Testing with Pytest



Automated Testing with Pytest

Section overview

1 Assertions in Pytest

1. Using the `assert` statement for checks
2. Comparing floats with `pytest.approx`
3. Testing for exceptions with `pytest.raises`

2 Configuring Pytest with `pyproject.toml`

3 Controlling test execution with markers

1. Skipping tests with `@pytest.mark.skip` and `@pytest.mark.skipif`
2. Handling expected failures with `@pytest.mark.xfail`
3. Organizing tests with custom markers

Automated Testing with Pytest

Section overview

4 Managing setup and teardown with fixtures

1. Defining fixtures with `@pytest.fixture`
2. Controlling fixture lifecycle with scopes (function, session)
3. Sharing fixtures across tests with `conftest.py`

5 Writing data-driven tests with parametrization

1. Using `@pytest.mark.parametrize` to run a test with multiple inputs

6 Isolating tests with mocking

1. Introduction to `unittest.mock.patch`
2. Using the `mock` fixture
3. Configuring mocks with `return_value` and `side_effect`
4. Understanding the difference between `Mock` and `MagicMock`

Python for DevOps

Structuring Python Projects



Structuring Python Projects

Section overview

1 Python modules and the import system

1. Understanding modules as single files
2. Using `import` and `from ... import`
3. How Python finds modules with `sys.path`

2 Organizing code with packages

1. The role of the `__init__.py` file
2. Importing from modules within a package

3 Subpackages and advanced imports

1. Creating nested package structures
2. Absolute vs. relative imports
3. Understanding common import errors and gotchas

Structuring Python Projects

Section overview

4 Running code from packages

1. Running modules from the command line with `python -m`
2. Separating library code from runnable scripts

5 Creating installable packages

1. Defining project metadata with `pyproject.toml`
2. Creating command-line scripts with entry points
3. Performing an editable install with `pip install -e .`

6 Structuring tests for a multi-file project

1. Organizing tests in a `tests/` directory
2. Importing application code into tests

Python for DevOps

CI/CD for Python



CI/CD for Python

Section overview

1 CI/CD and project overview

1. Understand the different steps in a CI/CD pipeline and their importance
2. Implement a sample Python application

2 Starter GitHub Actions workflow for Python projects

1. Implement a starter workflow to setup Python
2. Add steps to install dependencies and verify installation

3 Linting, formatting, static type, and security checks

1. Add and configure linting and formatting tools
2. Add and configure static type and security checks
3. Integrate checks into our GitHub Actions workflow

CI/CD for Python

Section overview

4 Integrate automated testing into our workflow

1. Add a basic test suite with Pytest
2. Integrate automated tests into our pipeline
3. Migrate to a matrix strategy to ensure tests are successful in multiple Python versions

5 Publish our Python library to PyPI

1. Explore how to build our library, as well as the generated artifacts
2. Learn how to install libraries manually with `pip`
3. Extend our GitHub Actions workflow to publish the generated artifacts to both TestPyPI and PyPI
4. Test installing the published library with `pip`

Python for DevOps

Introduction to **click**



Introduction to **click**

Building easy-to-use CLIs with Python

- **click** is a popular Python library for creating beautiful and easy-to-use Command-Line Interfaces (CLIs).
- It uses decorators (@) to turn normal Python functions into CLI commands.
- Key Decorators:
 - `@click.command()`: The main decorator that marks a function as a CLI command.
 - `@click.option()`: Defines an optional flag or parameter, like `--verbose` or `--out <filename>`.
 - `@click.argument()`: Defines a required positional argument.
- **click** leverages decorators to automatically handle:
 - Parsing arguments from the command line.
 - Passing those values to your function's parameters.
 - Validating types.
 - Generating a helpful `--help` message automatically.

```
@click.command()
@click.argument("url")
@click.option("--timeout", default=5)
@click.option("--verbose", "-v", is_flag=True)
def ping(
    url: str,
    timeout: int,
    verbose: bool
):
    ...
```



```
> ping-cli https://www.google.com --timeout 10
```

- ✓ With a bit of additional setup, we can expose the CLI in a user-friendly way via executable binaries.

Python for DevOps

CI/CD Tooling Overview



CI/CD Tooling Overview

Understand the different tools and their use-cases in CI/CD pipelines

CI/CD Tooling Overview

Understand the different tools and their use-cases in CI/CD pipelines

CI/CD Tooling Overview		
Tool	Role	Function
git & GitHub	Version Control System & Platform	git tracks our code changes locally. GitHub hosts our repository and provides the infrastructure for automation (GitHub Actions). The entire CI/CD process is triggered by Git events like push or rele ase.

CI/CD Tooling Overview

Understand the different tools and their use-cases in CI/CD pipelines

CI/CD Tooling Overview		
Tool	Role	Function
<code>git</code> & GitHub	Version Control System & Platform	<code>git</code> tracks our code changes locally. GitHub hosts our repository and provides the infrastructure for automation (GitHub Actions). The entire CI/CD process is triggered by Git events like <code>push</code> or <code>rele</code> ase.
<code>pyproject.toml</code>	Project Configuration File	Central file to configure all the tools and define project metadata.

CI/CD Tooling Overview

Understand the different tools and their use-cases in CI/CD pipelines

CI/CD Tooling Overview		
Tool	Role	Function
<code>git</code> & GitHub	Version Control System & Platform	<code>git</code> tracks our code changes locally. GitHub hosts our repository and provides the infrastructure for automation (GitHub Actions). The entire CI/CD process is triggered by Git events like <code>push</code> or <code>release</code> .
<code>pyproject.toml</code>	Project Configuration File	Central file to configure all the tools and define project metadata.
<code>black</code>	Code Formatter	Enforces a consistent style, failing the build if any file needs reformatting.

CI/CD Tooling Overview

Understand the different tools and their use-cases in CI/CD pipelines

CI/CD Tooling Overview		
Tool	Role	Function
git & GitHub	Version Control System & Platform	git tracks our code changes locally. GitHub hosts our repository and provides the infrastructure for automation (GitHub Actions). The entire CI/CD process is triggered by Git events like push or release .
pyproject.toml	Project Configuration File	Central file to configure all the tools and define project metadata.
black	Code Formatter	Enforces a consistent style, failing the build if any file needs reformatting.
ruff	Linters	Performs static analysis for a wide range of issues beyond formatting, such as style violations, logical errors, and unsorted imports.

CI/CD Tooling Overview

Understand the different tools and their use-cases in CI/CD pipelines

CI/CD Tooling Overview		
Tool	Role	Function
<code>git</code> & GitHub	Version Control System & Platform	<code>git</code> tracks our code changes locally. GitHub hosts our repository and provides the infrastructure for automation (GitHub Actions). The entire CI/CD process is triggered by Git events like <code>push</code> or <code>release</code> .
<code>pyproject.toml</code>	Project Configuration File	Central file to configure all the tools and define project metadata.
<code>black</code>	Code Formatter	Enforces a consistent style, failing the build if any file needs reformatting.
<code>ruff</code>	Linters	Performs static analysis for a wide range of issues beyond formatting, such as style violations, logical errors, and unsorted imports.
<code>mypy</code>	Static Type Checker	Checks for inconsistencies in defined and provided types.

CI/CD Tooling Overview

Understand the different tools and their use-cases in CI/CD pipelines

CI/CD Tooling Overview		
Tool	Role	Function
<code>bandit</code>	Security Scanner	Scans for common security vulnerabilities, looking for patterns like hardcoded passwords, use of insecure modules, or dangerous subprocess calls.

CI/CD Tooling Overview

Understand the different tools and their use-cases in CI/CD pipelines

CI/CD Tooling Overview		
Tool	Role	Function
bandit	Security Scanner	Scans for common security vulnerabilities, looking for patterns like hardcoded passwords, use of insecure modules, or dangerous subprocess calls.
pytest	Testing Framework	Used for test automation. This is the crucial step that validates our code's logic and behavior , being one of the most important elements of CI/CD pipelines.

CI/CD Tooling Overview

Understand the different tools and their use-cases in CI/CD pipelines

CI/CD Tooling Overview		
Tool	Role	Function
bandit	Security Scanner	Scans for common security vulnerabilities, looking for patterns like hardcoded passwords, use of insecure modules, or dangerous subprocess calls.
pytest	Testing Framework	Used for test automation. This is the crucial step that validates our code's logic and behavior , being one of the most important elements of CI/CD pipelines.
build	Package Builder	Reads pyproject.toml and creates the standard, distributable package files: the source distribution (.tar.gz) and the wheel (.whl).

CI/CD Tooling Overview

Understand the different tools and their use-cases in CI/CD pipelines

CI/CD Tooling Overview		
Tool	Role	Function
bandit	Security Scanner	Scans for common security vulnerabilities, looking for patterns like hardcoded passwords, use of insecure modules, or dangerous subprocess calls.
pytest	Testing Framework	Used for test automation. This is the crucial step that validates our code's logic and behavior , being one of the most important elements of CI/CD pipelines.
build	Package Builder	Reads pyproject.toml and creates the standard, distributable package files: the source distribution (.tar.gz) and the wheel (.whl).
twine	Package Uploader	Used behind the scenes by pypa/gh-action-pypi-publish , ensures the secure upload of our package files.

CI/CD Tooling Overview

Understand the different tools and their use-cases in CI/CD pipelines

CI/CD Tooling Overview		
Tool	Role	Function
bandit	Security Scanner	Scans for common security vulnerabilities, looking for patterns like hardcoded passwords, use of insecure modules, or dangerous subprocess calls.
pytest	Testing Framework	Used for test automation. This is the crucial step that validates our code's logic and behavior , being one of the most important elements of CI/CD pipelines.
build	Package Builder	Reads pyproject.toml and creates the standard, distributable package files: the source distribution (.tar.gz) and the wheel (.whl).
twine	Package Uploader	Used behind the scenes by pypa/gh-action-pypi-publish , ensures the secure upload of our package files.
python-semantic-release	Versioning and Release	Automatically bumps the package version and creates a new GitHub Release based on the commit history and messages.

Python for DevOps

Target Workflow Overview



Target Workflow Overview

Understand the bigger picture of the workflow we are trying to achieve

